

Simulation Instantiation Module (SIM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: SIMULOS® — Simulation Governance Architecture

Parent Standard: Simulation Execution Standard (SES)

Operational Layer: Simulation Execution Governance Layer

Category: Governance & Enforcement

Subcategory: Simulation Governance

Type: Parent Standard Module

Governed Space: Simulation Instantiation

Version: 1.0

Status: Canonical · Open Module

Effective Date: 19 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · ASGA™ · RIS™

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Governed Space

Simulation Instantiation

Canonical Definition

Simulation Instantiation Module (SIM) defines the governance framework through which a governed simulation specification is transformed into a specific executable Simulation Instance by binding the applicable Simulation Object, object version, model, environment, scenario, inputs, parameters, dependencies, randomization state, execution infrastructure, and other materially relevant conditions into an identifiable execution configuration before simulation activity begins.

SIM establishes the first internal governance space of the Simulation Execution Standard (SES).

Its purpose is to ensure that:

what was designed to be simulated

can be distinguished from:

what was actually assembled for execution.

SIM therefore establishes the exact governed starting configuration from which

subsequent Simulation Execution Control, Observation, Repetition, and Documentation proceed.

Operational Role

The preceding Simulation Scenario Standard (SSS) establishes:

which materially relevant conditions should be simulated.

SIM receives that governed scenario basis and asks:

What exact executable simulation instance has now been assembled from the approved simulation components and conditions?

The governed progression is:

Governed Simulation Basis

↓

Execution Requirement

↓

Simulation Instance Creation

↓

Object Binding

↓

Model & Environment Binding

↓

Scenario Binding

↓

Input & Parameter Binding

↓

Dependency & Infrastructure Binding

↓

Randomization / Initial-State Binding

↓

Instantiation Verification

↓

Executable Simulation Instance

↓

Simulation Execution Control

Module Operational Space

SIM governs: Simulation Instance identity, Simulation Execution identity
initialization, instantiation requirements, Simulation Object binding,
object-version binding, model binding, model-version binding, environment
binding, environment-version binding, scenario binding, scenario-version
binding, input-set binding, input-version binding, parameter binding,
parameter-baseline binding, assumption reference binding where material,
dependency binding, software dependency binding, external-service binding,
hardware binding, runtime binding, infrastructure binding, configuration
binding, randomization binding, seed binding, initial-state binding,
initial-condition binding, actor-population binding, agent-state binding,
temporal starting-state binding, execution-resource binding, execution
authorization reference, instantiation consistency,
intended-versus-instantiated configuration comparison, configuration conflict
identification, missing-component identification, unexpected-component
identification, instantiation completeness, instantiation status,
instantiation versioning, and Simulation Instance traceability.

Module Function

SIM exists because a simulation may be perfectly designed at the architectural
level while the actual executable instance differs materially from that
design.

Possible causes include: wrong model version, wrong environment version,
outdated Scenario Set, wrong Input Set, missing dependency, incorrect
parameter baseline, unexpected default values, changed software library,
changed hardware, unavailable external service, wrong randomization
configuration, incorrect initial state, different agent population, unrecorded
infrastructure change, or manual configuration error.

SIM prevents those differences from remaining invisible. Simulation
Specification ≠ Simulation Instance

SIMULOS® establishes a fundamental distinction:

Simulation Specification

describes the governed simulation that is intended to be executable.

Simulation Instance

represents the exact materially relevant configuration actually assembled for a particular execution.

Therefore:

Intended Configuration $\neq$ Instantiated Configuration by default.

The relationship must be established.

Simulation Instance

A Simulation Instance is the identifiable and execution-ready realization of a governed simulation configuration assembled for one Simulation Execution or defined group of executions.

Conceptually:

Simulation Object Model Environment Scenario Inputs Parameters Dependencies
Initial State Execution Infrastructure

=

Simulation Instance Simulation Instance Identity

Every materially relevant Simulation Instance should be sufficiently identifiable.

The identity may include: Simulation Instance ID, planned Execution ID, Simulation Object ID, object version, model version, environment version, Scenario ID, Scenario Set version, Input Set ID, parameter baseline, randomization configuration, dependency state, execution infrastructure, instantiation timestamp, and instantiation status.

Instance Identity $\neq$ Scenario Identity

A Scenario identifies the condition intended for examination.

A Simulation Instance identifies the complete executable realization of that Scenario under a particular technical and governed configuration.

One Scenario may therefore produce many Simulation Instances.

For example:

Scenario S-021

may be instantiated as:

Instance I-021-A

Instance I-021-B

Instance I-021-C

under different: model versions, random seeds, Input Sets, environments, or execution infrastructures.

Execution Requirement

Instantiation should begin from an identifiable Execution Requirement.

The requirement may originate from: Scenario Set, simulation campaign plan, validation requirement, engineering inquiry, autonomous follow-up simulation, incident analysis, stress escalation, scenario discovery, or another governed simulation need.

The requirement should make clear:

Why is this simulation instance being created?

Instantiation Requirement

An Instantiation Requirement defines which governed components must be assembled before execution can begin.

This may include mandatory: object version, model version, environment version, scenario, Input Set, parameters, dependencies, initial conditions, and runtime conditions.

Object Binding

Every Simulation Instance should remain bound to the exact Simulation Object it represents.

Conceptually:

Simulation Instance → Simulation Object ID → Object Version

This prevents an execution associated with one object version from later being interpreted as if it represented another.

Object-Version Binding

Where the Simulation Object changes materially, the applicable instance should identify which object version was instantiated.

For example:

Spacecraft Configuration v4.2

must remain distinguishable from:

Spacecraft Configuration v4.3.

No Object Substitution Principle

SIMULOS® establishes:

A simulation execution must not silently substitute a materially different Simulation Object version while preserving the identity of the original execution basis.

Model Binding

A Simulation Instance should identify the exact Simulation Model configuration used.

This may include: model identifier, model version, component versions, active model modules, enabled capabilities, model configuration, and relevant model dependencies.

Model-Version Binding

Where multiple model versions exist, a Simulation Instance should not rely solely upon the statement:

“Latest model used.”

It should preserve the exact version instantiated.

No Latest-Version Ambiguity Principle

SIMULOS® establishes:

A dynamically changing reference such as “latest” is insufficient where exact model identity materially affects execution reconstruction.

Environment Binding

The Simulation Instance should identify the exact governed Simulation Environment in which the execution is instantiated.

This includes the environment version established under SMES.

Conceptually:

Simulation Instance → Simulation Environment Version Environment Configuration Binding

Materially relevant environment attributes may include: environment version, simulation engine, runtime configuration, physics engine, network model, terrain state, environmental model, agent environment, external interfaces, and dependency state.

Scenario Binding

Every Simulation Instance should bind to the Scenario or Scenario Configuration it is intended to execute.

Conceptually:

Simulation Instance → Scenario ID → Scenario Version Scenario Configuration Binding

Where the Scenario contains variation, edge-case, or stress configurations, the instance should preserve the exact applicable configuration.

For example:

Scenario: Autonomous Recovery

Variation: Communication Delay = 8 s

Edge: Power near autonomy-transition threshold

Stress: Navigation uncertainty elevated

These are materially different from the generic Scenario identity alone.

No Generic-Scenario Binding Principle

SIMULOS® establishes:

Binding an execution only to a broad Scenario name is insufficient where materially relevant Scenario Variation, Edge, or Stress conditions determine execution behavior.

Input Set Binding

Every materially relevant Simulation Instance should identify the Input Set it will consume.

Conceptually:

Simulation Instance → Input Set ID → Input Versions

This preserves the SIDS governance basis established before execution.

Input Version Binding

Where inputs change over time, the instance should preserve the exact versions selected.

A new dataset must not silently replace the dataset originally intended for a historical execution.

Input Availability Check

Instantiation may identify whether required inputs are:

Available

Conditionally Available

Substituted

Missing

or:

Unavailable.

Material input absence should be resolved before execution or preserved as an explicit Instantiation Deviation.

Input Substitution at Instantiation

Where an intended input is unavailable and another is substituted, the relationship should remain visible.

Conceptually:

Required Input A

↓

Unavailable

↓

Substituted Input B

↓

Instantiation Deviation / Approved Substitution

The substitute does not become the originally required input merely because execution proceeds successfully.

Parameter Binding

A Simulation Instance should preserve the applicable parameter configuration established under SMES and the relevant Scenario Variation.

Parameter binding may include: baseline parameters, Scenario parameters, control parameters, runtime limits, threshold values, physical constants, behavioral parameters, and configuration switches.

Default Parameter Governance

Simulation software may contain default values.

SIM establishes:

A default value is still part of the instantiated configuration where it can materially influence execution.

Material defaults should not disappear merely because they were not manually entered.

No Invisible Default Principle

SIMULOS® establishes:

A simulation must not treat materially influential default configuration as if no configuration decision existed.

Assumption Reference Binding

SAS governs Simulation Assumptions.

SIM does not re-govern those assumptions.

Where an assumption materially conditions execution, however, the Simulation Instance may preserve a reference to the applicable assumption basis.

Dependency Binding

Simulation execution may depend upon external or internal systems including: libraries, APIs, databases, AI models, services, operating systems, middleware, simulation engines, communication services, external models, or hardware.

SIM should bind material dependencies to the Simulation Instance.

Dependency State

A dependency may be:

Available

Degraded

Restricted

Emulated

Substituted

Unavailable

or:

Dependency State Undetermined.

The instantiated state may materially affect execution.

External Service Binding

Where execution depends upon an external service, SIM may preserve: service identity, version where available, configuration, API state, endpoint behavior, availability, and relevant execution conditions.

No External-Service Assumption Principle

SIMULOS® establishes:

An external dependency must not be assumed materially unchanged merely because the simulation configuration itself was not modified.

Software Binding

A Simulation Instance may preserve materially relevant: software build, simulator version, compiler, libraries, runtime, container image, orchestration layer, operating system, or another technical state.

SIM does not prescribe how deeply every software component must be recorded.

Governance depth should reflect materiality.

Hardware Binding

Where hardware may materially influence execution, SIM may bind: processor type, accelerator, embedded hardware, sensor emulator, actuator hardware, hardware-in-the-loop component, firmware version, or other physical components.

Execution Infrastructure Binding

Simulation behavior may depend upon execution infrastructure.

Relevant infrastructure may include: workstation, cloud environment, HPC cluster, distributed simulation environment, hardware-in-the-loop bench, robotic test platform, mission simulator, or another execution environment.

Infrastructure Identity

SIM may preserve:

Simulation Instance → Execution

Infrastructure ID

where infrastructure materially affects timing, performance, determinism, or execution behavior.

Randomization Binding

Where execution uses randomness, the Simulation Instance should identify the applicable randomization basis where material.

This may include: randomization method, seed, sampling distribution, agent initialization, stochastic environment, random event schedule, or other

stochastic configuration.

Seed Binding

Where applicable:

Simulation Instance → Random Seed

may support later repeatability or diagnostic analysis.

Seed governance does not imply that execution will necessarily be perfectly reproducible.

Initial-State Binding

Simulation behavior frequently depends strongly upon the initial state.

SIM therefore governs the explicit binding of materially relevant initial conditions.

These may include: system state, environment state, agent state, position, velocity, resource levels, memory state, mission phase, authority state, sensor state, or active dependencies.

No Initial-State Neutrality Principle

SIMULOS® establishes:

Two executions using the same Scenario and Model are not necessarily comparable if their materially relevant initial states differ.

Agent-Population Binding

Multi-agent simulations may require identification of: number of agents, agent types, agent versions, roles, goals, behavioral configurations, authority relationships, and initial positions or states.

Autonomous-Agent State Binding

Where autonomous agents contain persistent or adaptive state, the Simulation Instance may need to preserve: memory state, policy version, permission state, tool availability, authority level, active goals, or previous interaction history.

This becomes particularly important where nominally identical runs can diverge because agent state differs.

Temporal Starting State

SIM may establish the temporal position from which the simulation begins.

Examples include: mission time, operational time, historical timestamp, relative simulation time, system lifecycle state, or event position.

Instantiation Time

The time at which the Simulation Instance was created may itself be relevant where: dependencies change, external services change, data update, model versions change, or execution infrastructure is dynamic.

Planned vs Actual Instantiation

SIM distinguishes:

Planned Configuration

from:

Instantiated Configuration.

The two should be compared before execution where material.

Instantiation Delta

An Instantiation Delta represents the material difference between the intended simulation configuration and the configuration actually assembled.

Conceptually:

Actual Instantiated Configuration – Intended Configuration = Instantiation Delta
Instantiation Delta Examples

A delta may include: different model version, changed environment, substituted Input Set, parameter difference, missing dependency, different seed, infrastructure change, altered agent population, different initial state, or unexpected software version.

Instantiation Delta Status

A material Instantiation Delta may be:

Accepted

Conditionally Accepted

Requires Correction

Requires Reauthorization

Execution-Blocking

or:

Undetermined.

No Silent Delta Principle

SIMULOS® establishes:

A material difference between intended and instantiated configuration must not disappear merely because the simulation is technically capable of running.

Configuration Conflict

A Configuration Conflict exists where two required simulation components impose incompatible conditions.

Examples include: Scenario requires Model v4 but environment supports only Model v3, Input Set uses units incompatible with the selected model, parameter configuration violates environment limits, hardware version does not support the required execution state, or scenario assumes a dependency that is unavailable.

Conflict Resolution

A Configuration Conflict may be:

Resolved

Conditionally Resolved

Accepted as Limitation

Execution-Blocking

or:

Unresolved.

The resolution should remain traceable where material.

Missing Instantiation Component

A required component may be absent.

Examples include: missing input, missing parameter, unavailable dependency, absent model module, missing environment element, or unresolved initial state.

No Missing-to-Default Principle

SIMULOS® establishes:

A required but missing instantiation component must not silently become a default configuration unless that substitution is itself governed.

Unexpected Instantiation Component

A component may appear in the assembled Simulation Instance despite not being part of the intended basis.

Examples include: automatically loaded library, inherited configuration,

persistent agent memory, cached state, environment plugin, external tool, or additional subsystem.

Where materially influential, it should remain visible.

Configuration Inheritance

Simulation systems may inherit configuration from: templates, previous runs, campaign defaults, environment defaults, user profiles, persistent state, or parent configurations.

SIM requires material inherited configuration to remain traceable.

No Hidden Inheritance Principle

SIMULOS® establishes:

Inherited configuration is part of the Simulation Instance where it materially influences execution, even if it was not explicitly selected for the current run.

Simulation Instance Completeness

Instantiation Completeness asks:

Have all materially required components of the intended simulation been sufficiently assembled and identified for execution?

Possible statuses may include:

Instantiation Complete

Conditionally Complete

Partially Complete

Incomplete

or:

Instantiation Completeness Undetermined. Instantiation Completeness $\neq$ Execution Readiness

An instance may contain every required component while unresolved: conflicts, authorization, compatibility problems, or blocking deviations

still prevent execution.

Simulation Instance Readiness

A Simulation Instance may receive a readiness status such as:

Execution Ready

Conditionally Execution Ready

Execution Restricted

Not Execution Ready

or:

Execution Readiness Undetermined.

Instantiation Verification

Before transfer to Simulation Execution Control, SIM should verify that the instantiated configuration sufficiently corresponds to the governed Execution Requirement.

Verification may examine: identity, version, compatibility, completeness, dependencies, inputs, parameters, initial state, infrastructure, randomization, and material deltas.

Instantiation Verification $\neq$ Validation

Instantiation Verification confirms the consistency of the executable configuration with the governed simulation basis.

It does not establish that: the model is valid, the simulation is accurate, the output will be correct, or the system is safe.

Instance Freeze

Where required, an instantiated configuration may be frozen before execution.

An Instance Freeze prevents material configuration change between:

Instantiation Verification

and:

Execution Initiation

unless the change is explicitly governed.

No Pre-Execution Drift Principle

SIMULOS® establishes:

A verified Simulation Instance should not materially change before execution without creating an identifiable configuration change or successor instance.

Successor Instance

Where a material change occurs before execution, the system may create a successor instance.

Conceptually:

Instance I-001

↓

Material Configuration Change

↓

Instance I-002

rather than silently modifying I-001.

Simulation Instance Versioning

A logical Simulation Instance may require version distinction where configuration changes before execution or across campaign preparation.

Campaign Instantiation

Large-scale Simulation Campaigns may instantiate thousands or millions of related executions.

SIM may govern these through:

Campaign Configuration Run-Specific Configuration

The campaign basis can define common configuration while individual runs preserve material differences.

Campaign Template

A Simulation Campaign may establish a common template including: model version, environment version, infrastructure, common Input Set, scenario family, parameter ranges, randomization strategy, and logging requirements.

Run-Specific Instantiation

Each run may then instantiate its own: Scenario configuration, random seed, parameter values, actor population, initial state, or other run-specific attributes.

No Campaign-Level Identity Substitution

SIMULOS® establishes:

A Campaign ID must not replace run-level identity where materially different execution configurations exist inside the campaign.

Autonomous Instantiation

Simulation Instances may increasingly be created automatically by AI or autonomous systems.

For example:

Observed Simulation Anomaly

↓

AI identifies unexplored condition

↓

New Scenario Configuration

↓

Autonomous Simulation Instantiation

SIM remains applicable.

Where material, the instantiation record should preserve: initiating actor/system, selection basis, generated configuration, authorization state, and parent execution relationship.

Instantiation Authority

A Simulation Instance may be created by:

Human Operator

Automated Workflow

AI System

Autonomous Agent

Simulation Orchestrator

or another authorized actor.

The initiating actor does not necessarily own the authority to execute the simulation.

Instantiation and execution authorization remain separable.

No Autonomous-Origin Opacity Principle

SIMULOS® establishes:

Automatically generated Simulation Instances must remain traceable to the

process or system that generated their material configuration.

Instance Parentage

A new Simulation Instance may derive from: baseline run, previous execution, failed run, edge-case discovery, stress escalation, scenario expansion, or incident investigation.

SIM may preserve:

Parent Instance → Derived Instance

where this relationship materially explains why the new simulation exists.

Instantiation Lineage

A lineage may appear as:

Baseline Instance

↓

Variation Instance

↓

Edge-Case Instance

↓

Stress Instance

↓

Recovery Instance

This provides a structured relationship between related executions.

Point-in-Time Reconstruction

SIM should allow later questions such as:

What exactly was instantiated?

Which Simulation Object version was represented?

Which model version was loaded?

Which environment version was active?

Which Scenario configuration applied?

Which Input Set was selected?

Which parameters and defaults applied?

Which dependencies were active?

Which seed or randomization state applied?

What was the initial state?

Which execution infrastructure was selected?

Did the actual instance differ from the intended configuration?

Instantiation Traceability

The complete traceability chain may be represented as:

Execution Requirement

↓

Simulation Object

↓

Model

↓

Environment

↓

Scenario

↓

Input Set

↓

Parameters

↓

Dependencies

↓

Initial State

↓

Infrastructure



Randomization State



Simulation Instance



Execution ID Relationship to SOS

Simulation Object Standard (SOS) governs the identity and boundary of the Simulation Object.

SIM binds the applicable governed Simulation Object to an executable instance.

Relationship to SMES

Simulation Model & Environment Standard (SMES) governs the model and simulation environment.

SIM selects and binds their exact governed versions for execution.

Relationship to SIDS

Simulation Input & Data Standard (SIDS) governs which inputs may enter the simulation.

SIM records which exact governed Input Set is instantiated.

Relationship to SSS

Simulation Scenario Standard (SSS) governs Scenario Design and Scenario Coverage.

SIM transforms a selected governed Scenario configuration into an executable instance.

Relationship to SECM

SIM ends when a sufficiently governed Simulation Instance is ready for execution.

SECM — Simulation Execution Control Module then governs what happens once that instance begins running.

The handoff is:

Governed Simulation Instance



Simulation Execution Initiation

Relationship to INTEGROS®

INTEGROS® may govern integrity of: configuration records, instance records, software artifacts, dependencies, or other relevant objects.

SIM does not duplicate general integrity governance.

Relationship to ADIT®

ADIT® may later audit whether the instantiated simulation basis is sufficiently evidenced and traceable.

SIMULOS® owns the simulation-specific instantiation process itself.

Relationship to VALIDOS®

VALIDOS® may later evaluate whether simulation execution and its resulting record can support a Validation Purpose.

SIM does not determine validation status.

Module Boundary

SIM begins when a governed Scenario or Simulation Campaign requires transformation into an actual executable simulation configuration.

SIM ends when: the Simulation Instance has an identity, materially relevant components are bound, intended and instantiated configurations are compared, material conflicts and deltas are visible, required dependencies are established, initial state is defined, Instantiation Completeness is assessed, Execution Readiness is established, and the resulting instance can enter Simulation Execution Control.

SIM does not govern: active runtime control, observation during execution, repeated execution, final execution documentation, simulation-output interpretation, or validation.

Minimum Implementation Framework

1. Establish the Execution Requirement

Identify the Scenario, campaign, investigation, or other governed basis requiring execution. 2. Create Simulation Instance Identity

Assign a persistent Simulation Instance identifier and planned Execution identity. 3. Bind the Simulation Object

Identify the exact Simulation Object and version represented by the instance. 4. Bind Model, Environment, Scenario, Inputs, and Parameters

Establish the materially relevant governed configuration. 5. Bind Dependencies, Infrastructure, Randomization, and Initial State

Record the supporting conditions capable of materially affecting execution. 6.

Compare Intended and Instantiated Configuration

Identify material Instantiation Deltas, missing components, unexpected components, inherited configuration, and conflicts. 7. Assess Instantiation Completeness

Determine whether the materially required configuration has been assembled. 8. Establish Execution Readiness

Classify the Simulation Instance as: Execution Ready, Conditionally Execution Ready, Execution Restricted, Not Execution Ready, or Execution Readiness Undetermined.

9. Freeze or Version the Instance Where Required

Prevent silent configuration change before execution. 10. Preserve Instantiation Traceability

Maintain:

Execution Requirement → Simulation Instance → Bound Configuration → Execution ID

Governance Outputs

SIM may produce: Simulation Instantiation Requirement, Simulation Instance Identifier, Simulation Instance Record, Execution Requirement Record, Object Binding Record, Object-Version Binding, Model Binding Record, Model-Version Binding, Environment Binding Record, Environment-Version Binding, Scenario Binding Record, Scenario-Version Binding, Scenario Configuration Binding, Input Set Binding, Input Version Binding, Parameter Binding Record, Parameter Baseline Binding, Default Parameter Record, Assumption Reference Binding, Dependency Binding Record, External Service Binding, Software Configuration Binding, Hardware Configuration Binding, Execution Infrastructure Binding, Randomization Configuration Record, Seed Binding Record, Initial-State Record, Agent-Population Record, Autonomous-Agent State Record, Temporal Starting-State Record, Instantiation Authority Record, Configuration Inheritance Record, Instantiation Delta, Configuration Conflict Record, Missing Instantiation Component Record, Unexpected Instantiation Component Record, Instantiation Completeness Assessment, Simulation Instance Readiness Status, Instantiation Verification Record, Instance Freeze Record, Successor Instance Record, Simulation Instance Version Record, Campaign Instantiation Record, Run-Specific Instantiation Record, Simulation Instance Parentage Record, Instantiation Lineage Map, and Governed Simulation Instance.

Use Case 1 — Autonomous Vehicle Simulation Campaign

Scenario

An autonomous vehicle company runs a large simulation campaign involving a pedestrian-interaction edge case.

The intended configuration specifies: Perception Model v12.4, Environment v7.2, Scenario S-441, defined sensor-noise distribution, mapped pedestrian behavior, and a controlled Input Set.

One cluster of executions produces unusually strong collision-avoidance performance. Application

SIM reconstructs the instantiated configuration and identifies that the affected runs used:

Perception Model v12.5

because the simulation orchestrator automatically loaded the latest available model.

All other Scenario identifiers remained unchanged. Result

The organization does not mistakenly attribute the behavioral improvement to the Scenario or new control logic.

A material Instantiation Delta is identified:

Intended Model v12.4 → Instantiated Model v12.5

and affected executions remain separately traceable. Use Case 2 — Spacecraft Mission Simulation

Scenario

A space organization prepares autonomous recovery simulations for a spacecraft under: communication delay, navigation uncertainty, constrained power, and partial sensor degradation.

The mission model and Scenario are correctly defined.

However, a critical run uses an updated thermal-environment dependency that differs from the version used in the rest of the campaign. Application

SIM binds each execution to:

Spacecraft Configuration → Mission Model → Environment → Scenario → Inputs → Dependency State → Initial State → Infrastructure

and detects the materially different thermal-model dependency before execution results are aggregated with the broader campaign. Result

The organization can distinguish:

same mission Scenario

from:

same executable simulation basis.

The run may still be useful, but its configuration difference cannot disappear inside a large Monte Carlo campaign. Use Case 3 — Autonomous AI Agent Simulation

Scenario

An AI agent is simulated under a scenario involving: restricted tool permissions, delayed external responses, conflicting information, and autonomous task continuation.

Two apparently identical executions produce materially different behavior.
Application

SIM identifies that one Simulation Instance began with:

empty agent memory

while the second inherited:

persistent state from a previous simulation episode.

The scenario, model, tools, and explicit parameters were otherwise identical.
Result

The difference becomes an explicit Initial-State / Configuration Inheritance difference rather than being misclassified as unexplained model nondeterminism.

The organization can now determine whether persistent state should be deliberately included, excluded, or separately governed in future executions.

Architectural Position

Simulation Instantiation Module (SIM) is the first internal governance space of the Simulation Execution Standard (SES).

The complete SES progression is:

Simulation Instantiation

↓

Simulation Execution Control

↓

Simulation Observation

↓

Simulation Repetition

↓

Simulation Execution Documentation

SIM answers:

What exactly has been assembled for execution?

SECM then asks:

How must that Simulation Instance remain controlled while it runs?

SOM asks:

What materially relevant events occurred during execution?

SRPM asks:

How should repeated executions remain distinguishable and comparable?

SEDM asks:

Can the complete execution later be reconstructed?

SIM therefore establishes the execution starting-state governance layer upon which the rest of SES depends.

Foundational Principle

A simulation cannot be governed as executed merely because its intended configuration is known; the exact Simulation Object, model, environment, scenario, inputs, parameters, dependencies, initial state, and execution conditions actually assembled for the run must remain identifiable.

Canonical Closing Statement

Simulation Instantiation Module establishes the entry point into governed Simulation Execution by transforming a governed simulation specification into a specific, identifiable, and execution-ready Simulation Instance. Through object, model, environment, scenario, input, parameter, dependency, infrastructure, randomization, and initial-state binding; intended-versus-instantiated configuration comparison; configuration conflict and inheritance visibility; Instantiation Delta governance; completeness assessment; readiness determination; versioning; and complete instantiation traceability, SIM prevents intended simulation configuration from being mistaken for the configuration that actually entered execution and creates the controlled starting basis required for Simulation Execution Control, Observation, Repetition, Documentation, and downstream Simulation Output Governance.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>